# DLDCA Outlab: How to network and make friends

Week 3

August 26, 2026

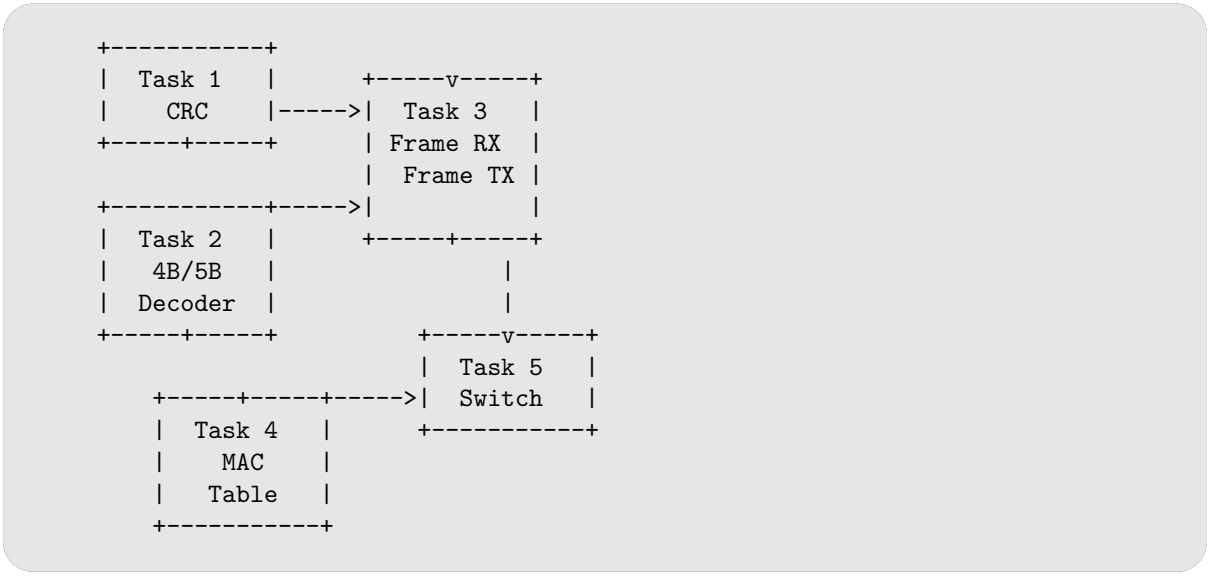
## Introduction

In this outlab, we will build a small Ethernet-inspired networking stack in Verilog. Rather than implementing an entire network protocol, we will implement a few simplified pieces of a Layer 1/2 datapath and gradually combine them into a working multi-port switch.

The tasks are designed to build on one another. Task 1 implements CRC for data integrity, while Task 2 implements the 4B/5B encoding used on the wire. Task 3 combines these components to implement frame transmission and reception. Task 4 implements a small MAC-learning forwarding table. Finally, Task 5 combines the frame and switching logic into a multi-port hardware switch.

### Task workflow

The dependency between the tasks is:




Task 1 and Task 2 implement basic building blocks. Task 3 uses these blocks to understand and process complete frames. Task 4 is independent of frame encoding and focuses on the forwarding decision made by a Layer 2 switch. Task 5 brings everything together: received frames are decoded and checked, the source address is learned, the destination address is looked up, and a valid frame is transmitted through the appropriate port.

## Necessary tools

We shall be using `icarus-verilog` (or) `iverilog` in order to compile verilog HDL (Hardware Description Language). To visualize waveforms we shall use the VS Code extension 'Vaporview'.

## Structure of submission/templates

Submission format:

```
your-roll-no/
|- task3/
    |- rx_frame.v (TODO)
    |- tx_frame.v (TODO)
    |- tb_task3.v
|- task4/
    |- switch_table.v (TODO)
    |- tb_task4.v
|- task1/
    |- crc.v (TODO)
    |- tb_task1.v
|- task2/
    |- encoder.v (TODO)
    |- decoder.v (TODO)
    |- tb_task2.v
|- task5/
    |- switch.v (TODO)
    |- tb_task5.v
```

## Tasks

## Task 3: Frame Receiver and Transmitter

In this task, you will build a complete link-layer frame on top of the CRC engine from Task 1 and the 4B/5B encoder/decoder from Task 2. The frame format is Ethernet-inspired and structured as follows:

### Frame Format

A frame on the wire has the following layout:

```
START | DST_ADDR | SRC_ADDR | PAYLOAD | CRC | END
```

Field widths and properties are detailed below:

| Field | Unencoded Width (bits) | Description | Encoded Width On-Wire (bits) |
|---|---|---|---|
| START | 5 | Marker: `11000` | 5 |
| DST_ADDR | 8 | Destination address | 10 |
| SRC_ADDR | 8 | Source address | 10 |
| PAYLOAD | 64 | Payload data | 80 |
| CRC | 8 | CRC-8 checksum | 10 |
| END | 5 | Marker: `11001` | 5 |
| **Total** | **94** | – | **120 (15 bytes)** |

- All fields between `START` and `END` are encoded using the 4B/5B scheme from Task 2.
- `START` (`11000`) and `END` (`11001`) are raw 5-bit wire markers and bypass 4B/5B encoding/decoding.
- The CRC is computed over `DST_ADDR`, `SRC_ADDR`, and `PAYLOAD` (80 bits total) in that order, using the CRC-8 parameters from Task 1 (`POLY=8'hD5`, `INIT=8'hFF`). The CRC field itself is not included in the CRC computation.
- Although individual 4B/5B symbols are 5 bits wide (not byte-aligned), the complete frame spans exactly 120 bits (**15 bytes**).

## Part A: Frame Receiver

Implement `rx_frame.v`. The receiver accepts a byte-wide wire stream and reconstructs valid frames.

```
module rx_frame (
    input             clk,
    input             rst,
    input      [7:0]  data_in,
    input             data_valid,

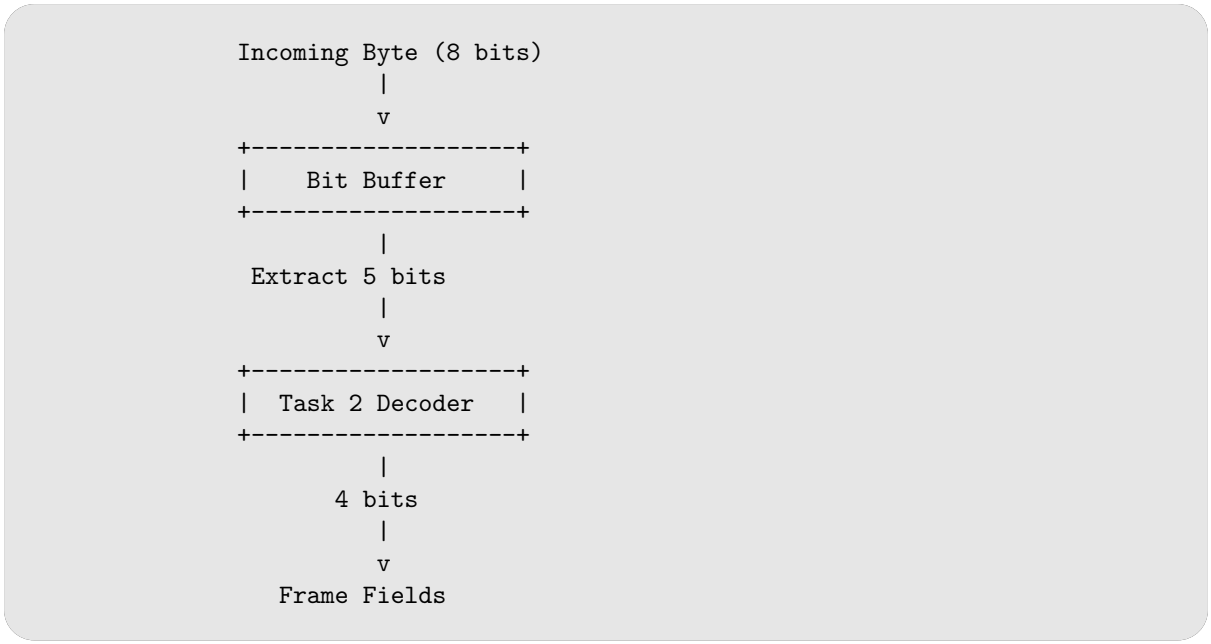
    output reg        frame_valid,
    output reg [7:0]  dst_addr,
    output reg [7:0]  src_addr,
    output reg [63:0] payload
);
```

When `data_valid` is asserted, `data_in` contains 8 incoming wire bits. Because 4B/5B symbols are 5 bits wide, 5-bit symbol boundaries will shift relative to incoming byte boundaries.

**A basic stub file has been provided to help you out.**

### Bit Buffering

To handle this alignment mismatch, maintain a bit buffer. Incoming bytes are appended to the buffer, and whenever at least 5 undecoded bits are available, the next 5-bit symbol is extracted and decoded.




### Receiver State Machine

Because the frame length is fixed, you can implement a clean 3-state machine using a single 4B/5B decoder and a nibble counter:

| State | Action | Next State |
|---|---|---|
| IDLE | Search bit buffer for START (11000) marker | RX_DATA |
| RX_DATA | Decode 22 nibbles (11 bytes) sequentially. Stream the first 10 bytes to the CRC engine; store the 11th byte as the received CRC. | CHECK_END |
| CHECK_END | Verify END (11001) marker and match calculated CRC. Pulse frame_valid if successful. | IDLE |

If an invalid 5-bit symbol is encountered during decoding, abort and return to IDLE.

## Part B: Frame Transmitter

Implement tx_frame.v. It accepts raw frame fields and emits the encoded 15-byte wire stream.

```
module tx_frame (
    input             clk,
    input             rst,
    input             start,
    input      [7:0]  dst_addr,
    input      [7:0]  src_addr,
    input      [63:0] payload,

    output reg [7:0]  data_out,
    output reg        data_valid,
    output reg        busy
);
```

When start is asserted (while busy is low), set busy high and transmit the 120-bit frame byte-by-byte via data_out.

**Design Choice: CRC**

In `tx_frame.v`, instantiate the combinational module `crc_parallel` (with `DATA_WIDTH=80` and `crc_in=8'hFF`) directly, rather than using the sequential wrapper `crc_parallel_serial`. Think why.

**Transmitter Logic**

1. **Combinational Frame Assembly (Cycle 0):** When `start` is asserted, combine `dst_addr`, `src_addr`, and `payload` into an 80-bit vector. Pass this vector directly into `crc_parallel` to evaluate the 8-bit CRC combinational output. Encode all 11 bytes using Task 2's 4B/5B encoders, attach the `START` (`11000`) and `END` (`11001`) markers, and form the complete 120-bit encoded frame.
2. **Frame Latch & Busy Assertion (Cycle 1):** On the next rising clock edge, latch the 120-bit compiled frame into an internal shift register and set `busy <= 1'b1`.
3. **Sequential Byte Transmission (Cycles 1–15):** Stream out the 120-bit frame 8 bits at a time using `data_out` with `data_valid <= 1'b1` across 15 clock cycles. Return `busy` to `0` upon completing the 15th byte.

### Testing

Run the testbench with:

```
$ make task3
```

Simulation output is automatically verified using `verify_task3.py`, which checks generated logs against a golden reference model.

## Task 4: MAC Learning and Forwarding

A Layer 2 switch decides which port should receive a frame based on its destination address. It can learn this information by observing the source address of incoming frames.

Implement a small MAC learning table. Whenever a frame arrives on port $P$ with source address $A$, record

$$A \longrightarrow P.$$

When a frame has destination address $D$, look up $D$ in the table.

### Required module

Complete `switch_table.v`:

```
module switch_table #(
    parameter ADDR_WIDTH = 8,
    parameter PORT_WIDTH = 2,
    parameter TABLE_SIZE = 8
) (
    input                       clk,
    input                       rst,

    input  [ADDR_WIDTH-1:0]     src_addr,
    input  [ADDR_WIDTH-1:0]     dst_addr,
    input  [PORT_WIDTH-1:0]     ingress_port,
    input                       frame_valid,

    output reg [PORT_WIDTH-1:0] egress_port,
    output reg                  known_destination
);
```

On every cycle where `frame_valid` is high, learn `src_addr` on `ingress_port`. If the source address already exists in the table, update its port.

For a known destination, assert `known_destination` and output the corresponding port. For an unknown destination, deassert `known_destination`; `egress_port` is then don't-care.

The table may be implemented as a small array of registers. You may assume that the testbench will not require more than `TABLE_SIZE` distinct addresses.

The switch does not modify frame addresses or payload.

### Working with Register Arrays in Verilog

To implement the table, you will need to store multiple addresses and their corresponding ports. In Verilog, you can declare an array of registers (a 2D array of bits) by specifying the bit-width of each element followed by the number of elements (the array depth).

For example, to create arrays for the MAC addresses, ports, and a valid bit for each of the `TABLE_SIZE` entries:

```
    // Declaration syntax: reg [bit_width] name [array_depth];

    reg [ADDR_WIDTH-1:0] table_addrs [0:TABLE_SIZE-1];
    reg [PORT_WIDTH-1:0] table_ports [0:TABLE_SIZE-1];
    reg                  table_valid [0:TABLE_SIZE-1];
```

To read or write to these arrays, you index them just like in C/C++, typically using an `integer` variable inside a `for` loop within an `always` block.

Here is some basic boilerplate to help you structure the table initialization and indexing:

```
    integer i;
    reg     handled; // Flag to track if we found a match or an empty slot

    // Sequential logic for learning/updating the table
    always @(posedge clk) begin
        if (rst) begin
            // Reset all valid bits to 0
            for (i = 0; i < TABLE_SIZE; i = i + 1) begin
                table_valid[i] <= 1'b0;
            end
        end else if (frame_valid) begin
            // Verilog for-loops do not support 'break' statements.
            // You can use the 'handled' flag to track if you have already
            // updated an entry or filled an empty slot.
            //
            // handled = 1'b0;
            //
            // // Pass 1: Loop through the table to see if src_addr already exists.
            // // If it does, update the port and set handled = 1.
            //
            // // Pass 2: If !handled, loop again to find the first empty slot.
            // // (Remember to check !handled inside this loop's if-condition too,
            // // so you only fill ONE empty slot, not all of them!)
            //
            // TODO: Implement the learning logic here.
        end
    end

    // TODO: Add combinational logic (always @*) to look up dst_addr
    // and drive egress_port and known_destination.
```

### Testing

Run:

```
$ make task4
```

The testbench checks learning, updating, known destinations, and unknown destinations.

## Task 1: CRC Engine

CRC (Cyclic Redundancy Check) is used by real protocols such as Ethernet, USB, and HDMI to verify data integrity. The sender computes a checksum over a stream of bits and appends it to the data. The receiver recomputes the checksum and compares the two values.

You will implement a CRC engine in three parts.

**See the Testing section for how to verify your CRC output against the provided Python script.**

### The Algorithm

CRC is computed serially, one bit at a time. One iteration is:

```
feedback = crc[MSB] XOR incoming_bit

if feedback == 1:
    crc = (crc << 1) XOR POLY
else:
    crc = (crc << 1)
```

Start with `crc = INIT = ff...ff` (all 1s) and feed the message bits through the loop, MSB first. The final `crc` value is the checksum.

**Recall:** For an $N$-bit register, left-shift and append 0 using {`crc[N-2:0], 1'b0`}.

## Part A: CRC-8 (Fixed Width)

Implement the following two CRC-8 modules:

**width = 8 bits, polynomial = `8'hD5`, initial value = `8'hFF`.**

### `crc8_update` (combinational)

Implement one iteration of the CRC algorithm. The module takes the current CRC value and one input bit, and produces the next CRC value.

Use only `assign` statements.

```
module crc8_update (
    input  [7:0] crc_in,
    input        data_bit,
    output [7:0] crc_out
);
```

### `crc8_serial` (sequential)

Accumulate the CRC over a stream of bits. On each clock edge where `data_valid` is high, feed `data_bit` through `crc8_update` and store the result in the CRC register.

`data_last` indicates that `data_bit` is the final bit of the stream. When `data_last` is set, assert `crc_valid` for one cycle to indicate that the CRC computation is complete.

Reset the register to `8'hFF` when `rst` is asserted. The `crc` output must always reflect the current register value.

```
module crc8_serial (
    input        clk, rst,
    input        data_bit,
    input        data_valid,
    input        data_last,
    output [7:0] crc,
    output       crc_valid
);
```

**You must instantiate `crc8_update` inside this module and connect it to the CRC register.**

## Part B: Parametric CRC

A **parameter** allows one module to work with different widths. Generalise your Part A implementation as follows.

### `crc_update` (combinational, parametric)

Implement the same logic as `crc8_update`, but make `WIDTH` and `POLY` parameters.

Replace the hardcoded indices 7 and 6 with `WIDTH-1` and `WIDTH-2`.

```
module crc_update #(
    parameter             WIDTH = 8,
    parameter [WIDTH-1:0] POLY  = 8'hD5
) (
    input  [WIDTH-1:0] crc_in,
    input              data_bit,
    output [WIDTH-1:0] crc_out
);
```

### `crc_serial` (sequential, parametric)

Generalise `crc8_serial` by instantiating `crc_update` with the appropriate parameters.

Use an additional `INIT` parameter to specify the initial CRC value.

**Note:** {`WIDTH{1'b1}`} is Verilog's bit-replication syntax for a WIDTH-bit all-ones constant.

```
module crc_serial #(
    parameter             WIDTH = 8,
    parameter [WIDTH-1:0] POLY  = 8'hD5,
    parameter [WIDTH-1:0] INIT  = {WIDTH{1'b1}}
) (
    input              clk, rst,
    input              data_bit,
    input              data_valid,
    input              data_last,
    output [WIDTH-1:0] crc,
    output             crc_valid
);
```

**Note:** This problem does not require `genvar` or `generate`. Parametric indexing with `WIDTH-1` is sufficient. Not every parametric problem needs a generate block.

## Part C: Parametric & Parallel CRC

The implementations above process one input bit per clock cycle, so processing a 16-bit word requires 16 cycles.

Here, you will generate hardware that performs `DATA_WIDTH` CRC iterations in a single clock cycle. The input is a `DATA_WIDTH`-bit word, and its bits must be processed MSB first before updating the CRC register.

**Assume** `DATA_WIDTH` $\leq$ `WIDTH`**.**

You will implement two modules for Part C: a purely combinational parallel engine (`crc_parallel`) and a sequential wrapper module (`crc_parallel_serial`).

### `crc_parallel` (combinational)

Implement a combinational chain of `DATA_WIDTH` CRC iterations.

```
module crc_parallel #(
    parameter             WIDTH      = 8,
    parameter [WIDTH-1:0] POLY       = 8'hD5,
    parameter             DATA_WIDTH = 8
) (
    input  [WIDTH-1:0]      crc_in,
    input  [DATA_WIDTH-1:0] data,
    output [WIDTH-1:0]      crc_out
);
```

To do this, initialize a `genvar` and use a `generate-for` loop. Each iteration should instantiate one `crc_update` module and connect its output to feed the next iteration. Here is a guide to a simple implementation:

**Hint:** You can implement the parallel chain using an array of `WIDTH`-bit registers:

```
wire [WIDTH-1:0] crc_stage [0:DATA_WIDTH];
```

This creates an **array of `WIDTH` bit-wide registers**, where the registers themselves are indexed from 0 to `DATA_WIDTH`. Hence length of this array is `DATA_WIDTH + 1`.

Use `crc_stage[0]` for `crc_in` and `crc_stage[DATA_WIDTH]` for `crc_out`. Then, for each stage $i$, instantiate `crc_update` and wire it approximately as:

```
for i = 0 to DATA_WIDTH-1
                                  +-------------+
  crc_stage[i] (ith stage) ---> | crc_update | ---> crc_stage[i + 1] (next stage)
  data[DATA_WIDTH - 1 - i] ---> |             |
      (ith data bit from MSB)   +-------------+
```

Thus, each stage takes the CRC from the previous stage and processes the next data bit, MSB first. Final array is connected to `crc_out` and first array is connected to `crc_in`.

### `crc_parallel_serial` (sequential)

Instantiate `crc_parallel` inside a sequential module to accept `DATA_WIDTH` bits per clock edge.

```
module crc_parallel_serial #(
    parameter             WIDTH      = 8,
    parameter [WIDTH-1:0] POLY       = 8'hD5,
    parameter [WIDTH-1:0] INIT       = {WIDTH{1'b1}},
    parameter             DATA_WIDTH = 8
) (
    input                   clk, rst,
    input  [DATA_WIDTH-1:0] data,
    input                   data_valid,
    input                   data_last,
    output [WIDTH-1:0]      crc,
    output                  crc_valid
);
```

When `data_valid` is high, process all `DATA_WIDTH` input bits in one cycle and store the resulting CRC on the next clock edge. Pulse `crc_valid` when the word flagged with `data_last` is consumed.

#### Food for thought

The parallel unit processes `DATA_WIDTH` bits in one cycle, but the generated hardware still contains `DATA_WIDTH` levels of CRC computation. The signal must propagate through all of them before reaching the CRC register.

Therefore, increasing `DATA_WIDTH` reduces the number of clock cycles required, but increases the combinational path length within each cycle.

What is the equivalent idea for bit adders? Can this be compared to a **Ripple Carry Adder (RCA)** or a **Carry Lookahead Adder (CLA)**? Could CRC have a CLA-like structure as well?

### Testing

#### Manual verification

Run the verifier in interactive mode:

```
python3 task1/verifier.py
```

It will prompt you for the message, width, polynomial, and initial value, then print the expected CRC.

#### Automated testing

Use the Makefile:

```
$ make task1
```

This compiles your code and produces `task1.vvp`, then runs it once to show debug output. You can open the resulting `task1.vcd` in VaporView.

Finally, it runs the verifier in autograde mode:

```
python3 task1/verifier.py --autograde
```

A successful run will look like:

```
=== RUNNING CRC_VERIFIER.PY ===

Running task1.vvp...

TEST 1: PASS (MSG=AB, WIDTH=8, POLY=D5, INIT=FF, CRC=31)
...

3/3 CRC tests passed.
```

## Task 2: 4B/5B Encoder and Decoder

4B/5B is a line coding scheme in which every 4-bit data nibble is represented by a 5-bit codeword. The additional bit gives the transmitted stream useful properties for clock recovery and signal transitions.

In this task, implement the encoder and decoder as purely combinational logic. You should derive the output equations using Karnaugh maps (K-maps).

### The Encoding Scheme

Use the following standard 4B/5B data encoding table:

| Data | Code | Data | Code |
|---|---|---|---|
| 0000 | 11110 | 1000 | 10010 |
| 0001 | 01001 | 1001 | 10011 |
| 0010 | 10100 | 1010 | 10110 |
| 0011 | 10101 | 1011 | 10111 |
| 0100 | 01010 | 1100 | 11010 |
| 0101 | 01011 | 1101 | 11011 |
| 0110 | 01110 | 1110 | 11100 |
| 0111 | 01111 | 1111 | 11101 |

We will reserve the following two 5-bit values for framing:

| Marker | Code |
|---|---|
| START | 11000 |
| END | 11001 |

These values are not produced by the data encoder.

### Encoder

Complete `encoder.v`:

```
module encoder (
    input  [3:0] data,
    input        enable,
    output [4:0] encoded
);
```

The encoder is purely combinational. When `enable` is high, `encoded` must contain the codeword corresponding to `data`. The value of `encoded` when `enable` is low is don't-care.

Derive the five output Boolean functions using K-maps.

### Decoder

Complete `decoder.v`:

```
module decoder (
    input  [4:0] encoded,
    input        enable,
    output [3:0] data,
    output       valid
);
```

When `enable` is high, `valid` must be 1 exactly for the sixteen data codewords in the table. For a valid codeword, `data` must contain the corresponding 4-bit value. START, END, and all other 5-bit values are invalid data codewords.

The decoder is also purely combinational and should be implemented using K-map-derived logic.

### Invalid Input

As discussed in the inlabs, invalid inputs generally correspond to don't-cares. However it is better practice to set all outputs to zero in this case by doing something like.

```
assign output = valid ? out_reg : 4'b0000;
```

### Testing

The testbench checks all sixteen encoder mappings and the decoder's handling of valid and invalid codewords.

Run:

```
$ make task2
```

## Task 5: Multi-Port Layer 2 Switch

In this task, combine the frame receiver, frame transmitter, and MAC learning table into a four-port Layer 2 switch.

Each port carries the byte-wide encoded stream used by `rx_frame` and `tx_frame`. A valid received frame is decoded and checked before the switch makes a forwarding decision.

### Required module

Complete `switch.v`:

```
module switch (
    input         clk,
    input         rst,

    input  [7:0]  rx0_data,
    input         rx0_valid,
    input  [7:0]  rx1_data,
    input         rx1_valid,
    input  [7:0]  rx2_data,
    input         rx2_valid,
    input  [7:0]  rx3_data,
    input         rx3_valid,

    output [7:0]  tx0_data,
    output        tx0_valid,
    output [7:0]  tx1_data,
    output        tx1_valid,
    output [7:0]  tx2_data,
    output        tx2_valid,
    output [7:0]  tx3_data,
    output        tx3_valid
);
```

For every valid frame, the switch must learn the source address on the ingress port and look up the destination address.

If the destination is known, forward the frame only to the corresponding egress port. If the destination is unknown, flood the frame to all other ports except the ingress port.

Frames with an invalid CRC must be discarded and must not update the MAC learning table.

The transmitted frame must preserve the original source address, destination address, and payload. The switch only decides where the frame is sent.

You may assume that the testbench does not create simultaneous traffic that requires two different frames to use the same output transmitter. A frame may be dropped if its required output transmitter is busy.

### Testing

Run the standard test suite with:

```
$ make task5
```

The default testbench exercises MAC learning, known-destination forwarding, flooding, and rejection of frames with invalid CRCs. You can inspect the generated waveform (`task5.vcd`) to follow a frame through the complete receive–forward–transmit path.

#### Interactive Integration Testing

To simplify custom verification, an interactive test framework is provided in `interactive_tester.py`. You can specify your sequence of test commands in `task5/input.txt`:

```
# Syntax:
#   MAP <MACHINE> <PORT>      (Assigns a machine 'A'..'Z' to port 0, 1, 2, or 3)
#   SEND <SRC> <DST> <PAYLOAD> (Sends a frame with a 64-bit payload)

MAP A 0
MAP B 1
MAP C 2
SEND A B 0xAABBCCDDEEFF0011
SEND B A 0b1010101010101010101010101010101010101010101010101010101010101010
MAP A 3  # Overrides machine A to port 3
MAP D 1  # Overrides machine D to port 1
SEND A C 0x1234567890ABCDEF
SEND C D 0xFEDCBA9876543210
```

The framework operates in two distinct modes via command-line flags:

- `--setup-autograder`: Reads the command sequence from `task5/input.txt` (or a custom `--input-file`) to generate and inject the corresponding stimulus directly into the Verilog testbench (`task5/tb_task5.v`).
- `--run-autograder`: Executed after running the simulation, this mode parses the resulting simulation log (`task5_output.txt`) and verifies the observed frame switching against the expected behavior predicted by the reference model.